2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS) | 979-8-3503-6563-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/QRS62785.2024.00031

Symbolic Execution with Test Cases Generated by Large Language Models

Jiahe Xu[1], Jingwei Xu[1,*], Taolue Chen[2] and Xiaoxing Ma[1]
[1]State Key Lab of Novel Software Technology, Nanjing University, Nanjing, Jiangsu, China
[2]School of Computing and Mathematical Sciences, Birkbeck, University of London, UK
jiahexu@smail.nju.edu.cn, jingweix@nju.edu.cn, t.chen@bbk.ac.uk, xxm@nju.edu.cn
*corresponding author

Abstract—Symbolic execution is a powerful program analysis technique. External environment construction and internal path explosion are two long-standing problems which may affect the effectiveness and performance of symbolic execution on complex programs. The intrinsic challenge is to achieve a sufficient understanding of the program context to construct a set of execution environments which can guide the selection of symbolic states. In this paper, we propose a novel program-context-guided symbolic execution framework LangSym based on program's instruction/user manual. Leveraging the capabilities of natural language understanding and code generation in large language models (LLMs), LangSym can automatically extract the knowledge related to the functionality of the program, and generate adequate test cases and the corresponding environments as the prior knowledge for symbolic execution. We instantiate LangSym in KLEE, a widely adopted symbolic execution engine, to build a pipeline that could automatically leverage LLMs to boost the symbolic execution. We evaluate LangSym on almost all GNU Coreutils programs and considerable large-scale programs, showing that LangSym outperforms the existing strategies in KLEE with at least a 10% increase for line coverage.

Keywords–software testing; symbolic execution; large language model

1. Introduction

Software testing is an essential component of software development, crucial for ensuring that applications adhere to their specified requirements, operate correctly and provide a high-quality user experience. Software testing methods are typically divided into two main categories, i.e., black-box testing and white-box testing, differentiated by the necessity for access to the source code [1]. Black-box testing aims to validate the correctness of the software's functionality and ensure that it meets the software requirements. Specifically, black-box testing examines the relationship between inputs and outputs, as well as how the software responds to different inputs, while does not need to understand the internal structure or code of the software. In contrast, white-box testing processes are usually based on a detailed understanding of software's internal structure, algorithm design, code implementation, etc. Symbolic execution [2], [3], which is one of the most widely adopted program analysis techniques considered as a form of white-box testing, achieves wide successes in automatically generating high-coverage test cases and discovering bugs that hide deep in programs [4], [3]. Specifically, symbolic execution runs the program with symbolic input to analyze, for instance, what inputs cause each part of a program to execute. With its symbolic nature, the execution of the program is forked at every decision point (such as loops and conditional branching), generating new paths in the execution tree. Each path represents a sequence of operations performed by the program, conditioned on the symbolic inputs.

Having achieved great success in automated testing and verification, two long-standing issues, i.e., *external environment construction* (e.g., files or folders as the input) and *internal path explosion during execution*, remain unresolved [3]. Existing approaches primarily propose efficient path search strategies applied to the execution tree of symbolic execution to diminish the influence of path explosion, such as the program-independent searching strategies [5], [4], [6], [7], [8] and program-dependent approaches [9], [10]. However, program-independent strategies usually lack efficacy, while program-dependent approaches lack generalizability. The inputs related to external files or folders appear to be more critical in search space exploration. For instance, both *diff* and *ls* programs need files and/or the path of the folder as external inputs. Meanwhile, the executions are highly based on the content of the external inputs. Thus, external environment construction (e.g., creating folders and files with proper contents) is necessary when testing this type of program. Due to the extremely high complexity of traversing all proper external constructions, little work explores the progress of constructing the external environment to provide adequate files and folders as the external inputs for the program.

We believe that both external input and the internal execution tree are intrinsically connected when the program is developed: the former is treated as the static context and the latter as the dynamic context of the program. In practice, the two contexts can be extracted from the program accompanied by the functional requirements. From this perspective, these two issues could be handled in a unified way, boosting the efficacy and efficiency of symbolic execution.

In this paper, we propose a novel program-context-guided symbolic execution framework, LangSym, that extracts the context from the program to be the prior knowledge for symbolic execution. Specifically, LangSym uses LLM to read the program manual, which usually contains the functional description of the program, to generate adequate test inputs. Notice that these concrete test cases can be directly used as the program inputs. Leveraging the capabilities of natural lan-

2693-9177/24/$31.00 ©2024 IEEE
DOI 10.1109/QRS62785.2024.00031

guage understanding and code generation, LLM, with a proper prompt design, could generate the description for constructing the external environment. For example, LLM can generate the file content with the guidance of the program manual. Controlling the output format with a predefined domain-specific instruction, a postprocessing script could automatically construct the external environment for the symbolic execution. Thus, LangSym obtains a set of test cases that highly cover the functionality of the program. Then, we employ symbolic execution to generate additional test cases that provide more comprehensive coverage of the program. In this paper, we utilize GPT-4 [11], and KLEE [4] as our instantiations for the LLM and symbolic execution engine, respectively. Specific implementation details will be discussed in later sections. To show the efficacy of the proposed LangSym, we evaluate LangSym on the GNU Coreutils with 103 programs compared with popular existing strategies in KLEE. The results indicate a significant improvement in line coverage with our method. Furthermore, to validate the scalability, we select another seven relatively large programs from other GNU packages as test subjects and conduct the experiments. The results demonstrate that our method also performs relatively well on these programs.

In summary, we make the following contributions.

- We conceptualize a program-context-guided framework to handle the path explosion challenge in symbolic execution.
- We instantiate the framework with GPT-4 and KLEE to implement an automatic pipeline for program-context-guided symbolic execution.
- We carry out a thorough evaluation of the proposed method, showing case its efficacy and scalability across various GNU programs.

Paper organization. The background is introduced in Section 2. The proposed approach is presented in Section 3. The evaluation of the approach is given in Section 4, followed by some additional experiments in Section 5. The related work is discussed in Section 6. Finally, we conclude the paper in Section 7.

2. Background

In this section, we first present the basics of symbolic execution. We then introduce the most popular symbolic execution engine, KLEE, along with its seed mode, which is the key to integrating external knowledge into the symbolic execution process. Finally, we introduce Large Language Models (LLMs), especially their application for code-related tasks.

2.1 Symbolic execution

Symbolic execution is a program analysis technique that explores the potential execution paths of a program by using symbolic values instead of concrete input values [2], [3]. In symbolic execution, variables are treated as symbols rather than specific values, and program's control flow is traced through different paths, taking into account the symbolic representations of variables.

```
1 void foo(int a, int b) {
2     int x = 0, y = 1;
3     if (a == 0) {
4         y = x + 2;
5         if (b != 0)
6             x = 2 * (a + b);
7     }
8     assert (x != y);
9 }
```

Figure 1. Example C program

The process involves executing the program with symbolic inputs and creating symbolic expressions that represent the relationships between input variables and program variables. As the symbolic execution progresses, the tool or engine keeps track of the conditions under which different paths are taken, generating a set of constraints on the input symbols.

Symbolic execution is particularly useful for identifying and exploring different execution paths, including rare or hard-to-reach ones, without the need to run the program with all possible input values. This technique is widely used in various areas of software engineering, such as program testing, bug finding, and security analysis.

There are challenges associated with symbolic execution, including the potential for path explosion and the difficulty of handling complex programming constructs [3]. Despite these challenges, symbolic execution has proven to be a valuable tool for analyzing and understanding the behavior of software systems, aiding in identifying bugs and vulnerabilities and enhancing overall software reliability.

2.2 KLEE overview

KLEE is one of the most widely used open-source symbolic execution tools [4]. In KLEE, the execution paths of a program are represented as states, each primarily comprising symbolic memory, a program counter, and constraints. KLEE manages a queue of states to represent the explored execution paths. Initially, the queue contains only one state, where the program counter corresponds to the program's entry, and the constraints are empty, representing the beginning of the program.

The main operational process of KLEE is a loop. In each iteration, the engine selects a state from the queue and executes it according to its program counter. If the current instruction is a regular computational instruction, the state's symbolic memory is adjusted based on the instruction. In the case of a conditional branch instruction, the state will be forked into two states. One state's constraints are extended with the true branch's condition, while the other's constraints are extended with the false branch's condition. When the current instruction is an exit instruction, the state's constraints are submitted to a constraint solver. The solution generated by the solver replaces the initial symbolic values, serving as a concrete input for the program.

We use a simple C program (Figure 1) as an example to illustrate the running process of KLEE. KLEE performs symbolic

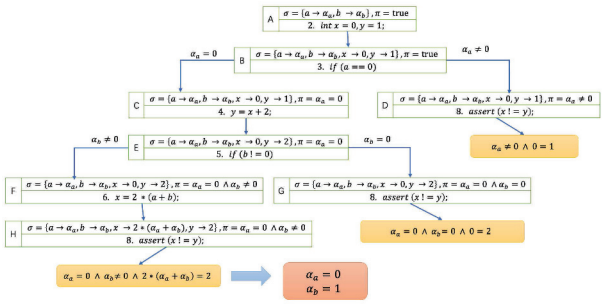

Figure 2. Execution tree of KLEE

execution on this program, and the resulting execution tree is depicted in Figure 2. Each path from the root node to a leaf node in the tree represents a possible execution path of the program. Each block represents a state, consisting of symbolic memory σ, condition expression π that should be satisfied at that program point, and a program counter pointing to the next instruction.

2.3 KLEE in seed mode

The traditional execution of KLEE starts from scratch without any external information or guidance. In newer versions of KLEE, a seed mode has been introduced, allowing users to provide concrete inputs of a program as initial seeds to guide the symbolic execution process. Each seed corresponds to a specific execution path of the program. In seed mode, the KLEE scheduler prioritizes the execution of states that satisfy the conditions of the seed path, essentially attempting to explore a path corresponding to the seed from the program's entry point. While exploring this path, states forked by branch instructions are also saved in the queue, representing paths adjacent to the seed path. Once the seed path has been fully explored, KLEE reverts to the regular execution mode and further explores the states in the queue.

Again, we use the program in Figure 1 as an example to briefly introduce the operation of KLEE in the seed mode. Firstly, concrete inputs need to be provided as seeds. For the "foo" program, we can take `a=0, b=0` as the initial seeds. This input corresponds to the "A-B-C-E-G" path in Figure 2. In the seed mode, when KLEE explores down from the initial state (the root of the tree), it prioritizes states that satisfy the seed conditions. Therefore, KLEE will first traverse the "A-B-C-E-G" path. After completing this path, the sibling nodes D of C and F of G are saved in the queue, since sibling nodes are forked together by their parent nodes. Subsequently, KLEE continues exploring the remaining parts of the execution tree starting from nodes D and F.

2.4 Large language models

Large language models (LLMs) such as Generative Pretrained Transformer (GPT) are advanced artificial intelligence systems designed to understand, generate, and interact with human language at a sophisticated level [11], [12]. These models are trained on vast amounts of text data, enabling them to grasp the nuances, complexities, and patterns of language. The development of LLMs has significantly influenced natural language processing (NLP) and artificial intelligence research, opening up new possibilities for human-computer interaction. LLMs are playing an increasingly important role in software development as well [13], [14]. Their ability to understand and generate human-like text extends to programming languages, enabling them to assist in a variety of software development processes. There are LLMs specifically fine-tuned for code-related tasks, for instance, Codex [15], AlphaCode [16] and Code Llama [17]. General-purpose LLMs, such as GPT-4, also possess good code understanding and generation capabilities [11]. LLMs have been applied to many code-related tasks, including code generation [18], [19], test case generation [20], [21], [22], bug detection and fixing [23], [24], etc.

3. Approach

In this section, we present our proposed approach LangSym. We give an overview of LangSym with an illustrative example of carrying out test case generation on the Linux program `cat`.

First, the program documentation is obtained by using the "–help" flag. The document generally includes the functionality of the program, basic usage, optional parameters, etc. A sample of the documentation of the program `cat` is shown in Figure 4.

Next, LangSym feeds the program documentation to an LLM, requesting it to generate a series of test cases based on the context of the documentation. With its powerful natural language understanding and code generation capabilities, the LLM can comprehend the program behavior described in the documentation, and generate a set of test cases that effectively cover the fundamental functionality and optional parameters of the program, as well as the files and/or folders to construct the external environment for execution. For instance, based on the description of usage and the presence of the "-A" argument in `cat`'s documentation, the LLM can generate a test case such as "`./cat -A file1`". Notice that the contents in "`file1`" are all generated by the LLM guided by the documentation.

LangSym then takes the LLM-generated test case as a seed and feeds it into a symbolic execution engine. The symbolic execution engine will symbolize the input arguments and file contents, representing them as undetermined values. The engine will then perform symbolic execution to generate additional test cases that explore different paths of the program. For example, with the test case "./cat -A file1" as the seed, after symbolic execution, the argument "-A" may lead to the generation of other arguments such as "-b", "-c", or invalid arguments such as "–", "-*". Similarly, "`file1`" can lead to files with different character contents.

In general, our approach involves feeding the program documentation to the LLM to generate test cases that cover the main functionalities of the program with executable external environments. Subsequently, these LLM-generated test

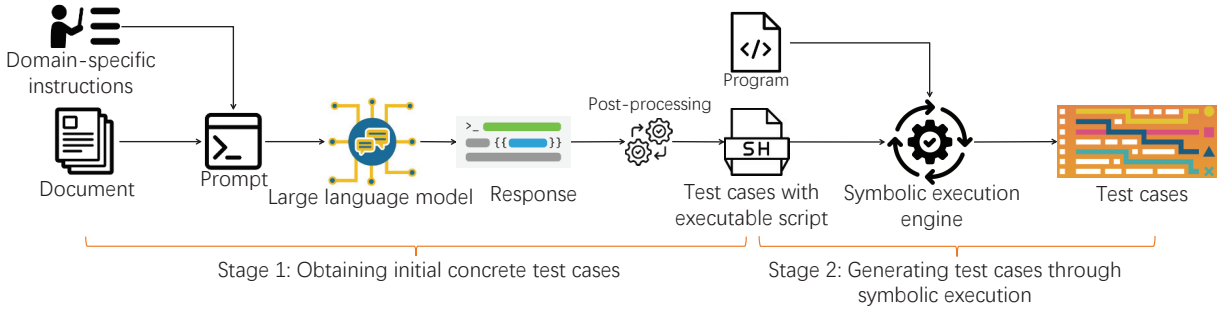

Figure 3. An overview of our approach LangSym

```
Usage: ./cat [OPTION]... [FILE]...
Concatenate FILE(s) to standard output.

With no FILE, or when FILE is -, read standard input.

  -A, --show-all           equivalent to -vET
  -b, --number-nonblank    number nonempty output lines, overrides -n
  -e                       equivalent to -vE
  -E, --show-ends          display $ at end of each line
  -n, --number             number all output lines
  -s, --squeeze-blank      suppress repeated empty output lines
```

Figure 4. Part of Linux cat manual

cases, along with the program source code, are provided to a symbolic execution engine, which symbolizes the inputs and performs symbolic execution, generating additional test cases aiming for higher code coverage. The workflow of our approach LangSym is illustrated in Figure 3. In the sequel, we present details of LangSym.

3.1 Initial concrete test cases acquisition

The first stage of LangSym is to obtain concrete test cases that can be directly used as program inputs. In this step, we employ a black-box testing method, generating test cases based on the program documentation. Additionally, the rise of LLMs has significantly reduced manual intervention in test case generation via documentation comprehension. LLMs possess strong text understanding capabilities and the ability to generate text and code, making them well-suited for generating test cases based on program documentation. With an appropriate prompt, an LLM can provide test cases that are almost ready for use. Users only need to design appropriate prompts when using LLMs. In this task, prompt design depends on the nature of the program under test to generate directly usable test cases. For example, when the program requires a file or a folder as input for testing, the LLM could describe the required file in natural language but not directly generate files in the operating system. To address this issue, we design domain-specific instructions to align LLM's response with a specific format and perform additional post-processing to obtain a directly usable test case. Specifically, to handle file construction, LangSym instructs the LLM to generate file creation commands with the following instructions.

```
instructions = "Please read the manual of a
  program. The manual's content is:\n" + manual +
  "\n." + \ "Then generate some testcases for the
  program based on the manual, to achieve high
  coverage. " + \ "Please only output test cases
  in the form of executable command, without
  giving extra information. " + \ "If the testcase
   contains files, also give the concrete content
  inside the file. " + \ "The file content should
  be provided by 'echo' " + \ "in the form like
  echo 'content of file' > file." + \ "If the
  testcase contains more than one command, use &&
  to concat them and provide in one line"
```

Based on the above instructions, we control the LLM to generate shell commands following the predefined output format: "`echo 'file content' > file`" when a file needs to be created. This instruction is then concatenated with subsequent shell script commands.

Using the Linux program `cat` as an example, by reading the manual from `cat` shown in Figure 4, the LLM can generate test cases such as "`echo 'hello world' > file1 && ./cat file1`", which can be executed by `cat` directly. Notice that LLMs hardly respond with the pure shell scripts without any explanation or description. Thus, post-processing is usually necessary to extract the shell scripts from LLM's response. A set of test cases for the program `cat` generated by LLM is listed below:

```
echo 'This is FileA content' > FileA && ./cat FileA
echo 'First file content' > File1 && echo 'Second
    file content' > File2 && ./cat File1 File2
echo 'Input from stdin' | ./cat
echo 'Start file content' > Start && echo 'End file
    content' > End && ./cat Start - End < <(echo '
    stdin content')
echo -e 'Line 1\nLine 2\twith tab\nNon-printing \
    x01character' > FileWithChars && ./cat -A
    FileWithChars
echo -e 'Line 1\n\nLine 3 after blank' >
    FileWithBlanks && ./cat -b FileWithBlanks
echo -e 'Line with end\nAnother line' > FileWithEnds
     && ./cat -E FileWithEnds
echo -e 'First line\nSecond line' > FileToNumber &&
    ./cat -n FileToNumber
echo 'Before stdin' > Before && echo 'After stdin' >
     After && ./cat Before - After < <(echo 'Content
     from stdin')
```

where each of the test cases is an executable command-line shell script. To apply various LLMs at this stage in LangSym,

we develop an interface that could execute API calls from the popular online LLMs such as GPT-4 [11] and Gemini Pro [25].

3.2 Test case generation through symbolic execution

Relying solely on test cases generated from program documentation is usually insufficient to comprehensively cover code blocks and paths of a program. Therefore, having obtained several specific test cases for programs in the previous stage, we use them as seeds and feed them to a symbolic execution engine, such as KLEE, which has a seed mode to run symbolic execution to generate more test cases.

Before feeding concrete test cases as seeds to KLEE, some conversion of the concrete test cases is necessary. Specifically, the concrete test cases obtained in the previous stage are executable command-line shell scripts, while KLEE has its own data structure to store seed inputs. Fortunately, KLEE provides a tool called "`ktest-gen`" for converting concrete inputs into its internal data structure. The caller needs to provide the input parameters and the external environment (e.g., files or folders) for this conversion.

Therefore, to obtain seeds that are suitable for KLEE, LangSym needs to extract input parameters from the test case and construct the external environment based on the command-line shell scripts. Here, we parse command-line shell scripts by predefined rules. Extracting input parameters is straightforward, as they can be directly obtained from the shell commands. The construction of the external environment mainly involves building the file system.

Taking the test case for `cat` in the previous paragraph as an example, the test case "`echo 'hello world' > file1 && ./cat file1`" consists of two commands. The first command is to create a file, and commands similar to this one are the commands to construct the external environment for the program `cat`. The second command is the shell script for testing the program, from which input parameters can be extracted (in this case, there are no parameters). In a clean directory, we parse executable command-line instructions. First, we consider shell script commands that do not involve the test target to be auxiliary commands for building the external environment. These shell commands are directly executed in the clean directory to obtain the file system environment required for testing. For commands that involve the test target, we do not execute them but instead perform string parsing to extract the input parameters. Finally, since we need to handle a large number of test cases, cleanup after parsing is necessary to restore the current directory to its initial clean state.

Once we obtain properly formatted seeds, we can feed them to KLEE and perform symbolic execution in its seed mode. This process aims to generate new test cases that cover more program paths, expanding the test coverage.

4. Evaluation

In this section, we evaluate LangSym using multiple benchmarks with respect to two main questions:

- RQ1: Does LangSym improve code coverage compared to the existing strategies in symbolic execution?
- RQ2: Can LangSym overcome the problem of poor scalability of traditional symbolic execution?

4.1 Evaluation setup

We choose the GNU Coreutils program set as our basic test set, which comprises 103 programs with a total of 43,077 lines of executable code since the Coreutils is the most widely used test set in research that primarily utilizes KLEE as the testing tool [4], [6], [9], [10]. The programs in the Coreutils program set interact frequently with the operating system, and testing them can cover a wide range of common system-level operations, such as creating, reading, writing, traversing, and deleting files.

Additionally, to evaluate the scalability of our approach, we choose another 7 larger programs as the supplementary test set, which contains `diff`, `find`, `grep`, etc, each consisting of thousands of executable lines. The detailed lines of code are listed in Table I. Research work on symbolic execution usually ignores testing large-scale programs because symbolic execution shows a very poor performance on large-scale programs due to the *path explosion* problem. However, we want to know to what extent our approach can overcome the scalability problem, so we design the supplementary evaluation on the selected 7 widely used larger programs in Linux.

In our evaluation, we use code coverage as the main metric to measure the effectiveness of our approach. Code coverage is the most straightforward indicator reflecting the quality of test cases. We use the default search strategy in KLEE, which combines RANDOM-PATH and COVERAGE-GUIDED search strategy and is most recommended by researchers [4], for comparison with our approach. Notice that our approach mainly focuses on generating initial test cases as the seed for symbolic execution. It could be easily combined with other search strategies in symbolic execution. For the time budget to the compared method, we allocate 1 hour of symbolic execution time for each program. The 1-hour time parameter is commonly used in previous research based on KLEE [4], [6], [9], [10]. For our proposed approach, the time cost for symbolic execution is roughly $\#seeds \times minutes$ for each program. Since the $\#seeds$ is about 15 on average, our time cost is about one-fourth of the compared method.

4.2 Experiments on Coreutils

First, we conduct our primary experiment using the GNU Coreutils program set. The documentation for Coreutils programs can be obtained through the "–help" flag. Using the documentation, along with additional prompts, we resort to the GPT-4 API to get a series of command-line shell scripts that can be directly executed as test cases for the programs. Each program receives more than ten test cases that responded by GPT-4 on average. Executing these test cases covers 24,176 lines of executable code in total, resulting in an average line coverage rate of 56.12% for entire programs in the Coreutils set.

As mentioned in Section 3, the test cases in the form of command-line shell scripts are transformed into a format that

complies with KLEE's internal requirements. It is important to note that this transformation cannot achieve complete equivalence. KLEE, which is an implementation of the symbolic execution engine, has its own execution environment that differs from a typical operating system in some aspects. For instance, in the case of the file system, KLEE models the file system as a flat layer of directory, unlike the tree structure in a Linux system. This indicates that some directory-related functionalities, such as "`cp -r`", cannot be accurately represented in KLEE's execution environment. However, in general, the transformation can achieve basic equivalence. Utilizing the `klee-replay` tool provided by KLEE, it is possible to replay the test cases in the specific format in KLEE. Replaying the transformed test cases covers 23,771 out of 43,077 lines of executable code, with an average line coverage of 55.18%. Compared to the executable command-line inputs, there is a slight loss in coverage.

Next, the transformed LLM-generated test cases are used as seeds and fed into KLEE for symbolic execution to generate more comprehensive test cases. For each program, there are several seeds, so we allocate 1 minute of symbolic execution time for each seed. Each symbolic execution process is independent, allowing for parallel execution. After completing all executions, the generated test cases are replayed, resulting in an average line coverage of 60.81% (covering 26,196 out of 43,077 lines) across all programs.

In the experiment, we observe that while KLEE in seed mode attempts to prioritize finding a path satisfying the seed's conditions, the complexity of symbolic execution, especially difficulties in constraint solving, may result in the inability to explore the path represented by the seed. This implies that the seed itself might not be present in the test cases generated through symbolic execution. Therefore, we combine the initial seeds with the newly generated test cases from symbolic execution to form the final test case set. Replaying this test case set results in an average line coverage of 65.82% (28,354 out of 43,077 lines) across all programs in Coreutils program set.

To compare with our method, we run KLEE in the regular mode with the default search strategy, setting 1 hour of symbolic execution time for each program. In this mode, the test cases generated by KLEE achieve an average line coverage of 54.52% (23,486 out of 43,077 lines).

From the above experimental results, we can observe that, compared to pure symbolic execution, the symbolic execution guided by concrete test cases generated based on documentation performs better. In particular, the test coverage of the Coreutils program set is increased from 54.52% to 65.82%. Moreover, with concrete test cases as guidance, symbolic execution can cover the main modules of the program more promptly, improving efficiency.

4.3 Experiments on other programs

Scalability is one of the critical problems for traditional symbolic execution. As the size of a program increases, the number of program paths grows exponentially, leading to the *path explosion* problem and significantly degrading the performance of symbolic execution. Therefore, traditional symbolic execution often targets relatively small-scale test subjects. For example, in the Coreutils program set, there are only a few programs with over a thousand lines of code, and the largest program, which is `ls`, has only 2104 lines of code.

The main reason that limits the scalability of symbolic execution is that, when there are numerous program paths, traditional symbolic execution struggles to determine which paths can lead to unexplored sections, resulting in exploration confined to a small portion of the code. In contrast, our approach guides symbolic execution with initially LLM-generated test cases based on documentation. This allows the symbolic execution engine to explore the main parts of the program more purposefully, which can overcome the scalability problem of symbolic execution to some extent. To validate this, we select seven relatively large-scale programs and conduct experiments similar to those in the previous part. The coverage results are shown in Table I.

From the results in the table, it can be observed that our approach does not always achieve highly desirable outcomes when dealing with larger-scale programs. On the `find`, `grep`, and `sed` programs, our approach achieves the expected outcomes. It involves building upon the test cases for the main functionalities generated by GPT-4, employing symbolic execution to explore additional program paths, and generating more test cases to enhance code coverage. Although performing better than the original symbolic execution, our approach does not increase the code coverage further on the other four programs with the symbolic execution in seed mode, compared to directly testing the program based on LLM-generated test cases.

The reasons for the lower test case coverage obtained through symbolic execution can be attributed to two main factors. Firstly, there is a loss when converting the executable command-line shell scripts format of the test cases generated by GPT-4 into the form of input seeds accepted by the symbolic execution engine KLEE. Due to differences in KLEE's modeling of the operating system and the actual Linux systems, the execution of input seeds in KLEE's environment is not entirely equivalent to the original test cases in the Linux system. Secondly, due to the complexity of constraint solving, the paths corresponding to the seeds may not be fully explored during the symbolic execution process. In our experiments, we observe that when testing the `make` program in KLEE's seed mode, there are multiple instances of warnings indicating solver failures during constraint solving.

In summary, according to the results in Table I, our method achieves a noticeable improvement in coverage compared to pure symbolic execution. Although our approach still suffers from the two core issues of symbolic execution: the *path explosion* problem and the difficulty of constraint solving, Our approach has mitigated the scalability issues associated with symbolic execution to some extent and improved the performance of symbolic execution.

TABLE I
LINE COVERAGE WITH DIFFERENT METHODS, WHERE THE NUMBERS IN THE PARENTHESES IN THE FIRST COLUMN REPRESENT THE EXECUTABLE LINES OF CODE IN THE PROGRAM.

Prog	Line Coverage (%)		
	KLEE	GPT-4	KLEE with seed
coreutils(43077)	54.52	56.12	65.82
diff(3003)	21.21	36.06	28.50
find(3767)	19.94	39.66	66.76
gawk(20558)	9.70	19.28	20.64
grep(2134)	48.45	50.94	59.56
sed(2337)	16.13	39.15	40.78
make(9369)	16.51	42.82	37.86
patch(4390)	16.20	10.18	25.58

TABLE II
LINE COVERAGE WITH DIFFERENT LLMS

Prog	Line Coverage (%)	
	GPT-4	Gemini
coreutils(42924)	56.01	46.77
diff(3003)	36.06	10.99
find(3767)	39.66	37.11
gawk(20558)	19.28	13.41
grep(2134)	50.94	/
sed(2337)	39.15	26.23
make(9369)	42.82	39.50
patch(4390)	10.18	9.18

5. ADDITIONAL EXPERIMENTS

In this section, we conduct additional experiments aiming to have a better understanding of LLMs' capabilities. First, we apply different LLMs to complete the same test generation task. Then, we compare the test cases generated by LLMs and human experts. Last, we feed the source code to LLMs and observe their performance.

5.1 Applying other large language models

Currently, apart from the GPT series, several LLMs also claim to have powerful capabilities. We select Code Llama [17] which claims excellent performance in code-related tasks, and Gemini [25], a relatively new large model introduced by Google, as potential alternatives to GPT-4. These models are used to complete the task, i.e., to generate test cases based on documents as mentioned in the previous sections by which we can compare the performance of the three models. Note that we use the same prompt.

The Code Llama family includes multiple variants designed to address different tasks, including foundation models (Code Llama), Python specializations (Code Llama-Python), and instruction-following models (Code Llama-Instruct). We select the Code Llama-Instruct model because its functionality aligns most closely with our task requirements, which involve generating test cases for programs based on the natural language prompts we provide.

Unfortunately, when experimenting with different parameter configurations of the Code Llama-Instruct model, we find that a significant portion of the generated content does not consist of expected directly executable command-line shell scripts, which indicates that the Code Llama model does not perform well in following our instructions for this task.

Next, we choose the Gemini model to accomplish the same task. Fortunately, the results generated by Gemini are executable command-line shell scripts, which are directly usable as test cases. Therefore, we conduct experiments using the test cases generated by Gemini with the obtained coverage results shown in Table II.

In the experiments, the Gemini model fails to provide results for the `kill` program in Coreutils because it identifies `kill` as a potentially dangerous term. Additionally, Gemini cannot generate usable test cases for the `grep` program. Furthermore, across all tested subjects, the test coverage of the test cases generated by Gemini is consistently lower than those generated by GPT-4. Therefore, the experimental results indicate that GPT-4 is the most effective large language model in the current setting.

5.2 Large language models vs human expert

According to the previous experimental results, it is evident that LLMs can effectively understand user requirements, comprehend program documentation, and generate test cases of good quality. Our next question is to what extent current LLMs can replace human experts. To answer this question, we select 10 programs from the previous test subjects. As software testing experts, we manually write several test cases for each of these 10 programs by carefully reading and understanding each program's documentation. We conduct experiments to measure the coverage of these test cases on the programs, and the coverage results are shown in Table III.

From Table III we can see that the coverage achieved by the manually written test cases exceeds those generated by GPT-4 in 9 out of the 10 programs. This suggests that, despite the powerful synthesis capabilities of LLMs, there are still certain gaps and limitations in specific tasks, and they may not yet match the level of expertise demonstrated by human experts.

Further comparing the test cases generated by GPT-4 with the manually written ones, we observe that a primary reason limiting the coverage of GPT-4's generated test cases is the failure to cover all optional parameters listed in the documentation. This suggests that LLMs fundamentally rely on statistical information derived from vast amounts of known data to generate content, rather than possessing a genuine understanding of text like human experts. Consequently, their attention mechanisms may still overlook some important aspects that should have been considered.

5.3 Feeding source code to large language model

In our approach, we treat LLMs as black-box testers. This is because we believe that they possess powerful text understanding capabilities, thus understanding program documentation described in natural language is well within their capability.

TABLE III
LINE COVERAGE WITH LLM AND HUMAN

Prog	Line Coverage (%)	
	GPT-4	Human
base64(140)	87.14	90.00
cat(303)	66.67	71.29
df(845)	73.25	73.61
du(433)	57.74	75.06
ln(384)	60.67	73.96
ls(2104)	58.94	77.19
mv(231)	64.50	82.25
sha256sum(397)	65.99	63.22
diff(3003)	36.06	58.17
find(3767)	39.66	48.16

Upon discovering that LLMs can effectively generate test cases based on program documentation, we wonder whether these models can directly comprehend the source code of a program and generate test cases accordingly.

To answer this question, we conduct experiments using the Coreutils program suite. We directly feed the source code of Coreutils programs to GPT-4 and instruct it to generate test cases based on the source code. Unfortunately, GPT-4 only returns results in the desirable form for 16 out of 103 programs.

Next, we feed both the program documentation and source code to GPT-4, instructing it to generate test cases. At this time, GPT-4 successfully returns results for 102 out of 103 programs. The only program that does not yield results is `ls`, whose source code file exceeds 5,000 lines (including blank lines, comments, and non-executable code). The test cases generated by GPT-4 for the 102 programs achieve a line coverage of 55.19%, which is similar to the results obtained solely relying on documentation. Therefore, feeding both program code and documentation to LLMs does not seem to enhance their effectiveness in generating test cases, but increases their burden.

In summary, despite the code understanding capabilities of LLMs, feeding program source code for test case generation purposes appears to be too challenging currently, especially when the scale of the program is larger. Therefore, feeding program documentation to LLMs for this task turns out to be a reasonable option so far. But with the advancing capability of dealing with the long-context input, large language models may be able to directly understand the program from the source code and generate the corresponding useful test cases for the downstream applications.

6. RELATED WORK

6.1 Symbolic execution

Since the proposal of symbolic execution [2], the issue of path explosion has consistently been a key problem that limits its practicality [3]. In response to the problem of path explosion, lots of research has put forth various solutions to guide the searching process of symbolic execution (state searching) [5], [4], [6], [7], [8], [9], [10].

State searching means searching for states that can bring more revenue to explore first, in order to improve the exploration performance within a given time budget. EXE [5] uses a best-first search heuristic, which favours the state that runs the fewest number of times among all states blocked at some line. KLEE [4] provides several search heuristics, including dfs, bfs, random-path, and so on. The most recommended one is random-path, which manages a process tree and picks one state by walking randomly in the tree. Subpath-Guided Search [6] presents a new measurement called *subpath*, that can more effectively reflect the possibility of a state to touch unexplored parts of the program. [7], [8] adopt Monte Carlo tree search to guide the search in symbolic execution. [9] teaches the symbolic execution engine to learn a search heuristic automatically, from a large number of previous runs. LEARCH [10] trains a machine learning model to predict the score of a state, that represents its value to be explored.

The purpose of state searching is to prioritize the exploration of more valuable paths. However, existing state-searching methods lack guidance from high-level semantic information. They rely solely on internal features and statistical information of states, such as branch depth and covered code blocks, to measure their worthiness of exploration. These metrics are not only complex but also insufficient for predicting the future trajectory of states, resulting in the suboptimal effectiveness of these search strategies. In contrast, our approach first obtains specific inputs covering the main functionalities of the program as seeds. These seeds guide symbolic execution, ensuring that it prioritizes exploring the core parts of the program code. Our approach is not only easy to use but also highly effective.

Besides pure symbolic execution, many research works also combine symbolic execution with fuzzing techniques. [26] and [27] apply symbolic execution techniques to the mutation of initial inputs, by collecting path conditions of the initial inputs, reversing some parts of the path conditions, and then solving constraints to obtain new inputs. This technique is called white-box fuzzing [27] or concolic execution [3], [28], [29], where ”concolic” is the compound of ”concrete” and ”symbolic”. [30], [31], [32] apply symbolic execution to generate initial seeds, then feed these seeds into fuzzers to generate more inputs. [28], [29], [33] leverage fuzzing and symbolic execution in a complementary manner, applying symbolic execution when fuzzing process gets stuck, and using symbolic execution to explore only the paths deemed interesting by the fuzzer and to generate inputs for conditions that the fuzzer cannot satisfy.

Our approach is somewhat similar to white-box fuzzing, which uses symbolic execution to derive more inputs from initial inputs. However, with the support of large language models, our method can achieve automation and efficiency in obtaining initial inputs, resulting in higher efficiency and quality of test generation.

6.2 Large language models for software testing

Based on the powerful code understanding and generation capabilities of LLMs, there has been considerable research integrating them into software testing. [20], [21] apply LLMs to automated unit test generation. [22] investigates the application of LLMs in the domain of mobile application test script generation and migration. [34] presents a code-aware prompting strategy for LLMs in test generation to improve coverage. [35] uses LLMs to assist the exploration process of search-based software testing, helping the testing to escape coverage plateaus by generating new test cases when the coverage improvements stall.

The research work mentioned above, which utilizes LLMs to support software testing, either involves generating test cases for a specific domain using LLMs [20], [21], [22], or addresses some issues in traditional testing processes with LLMs [34], [35]. A common feature among them is that large language models play a central role in their approaches. This implies that the limitations of these methods can largely be equivalent to the limitations of the capabilities of large language models. However, the capabilities of large language models are still limited, although they are already quite powerful. Our experiments indicate that when a considerably large-scale program source code is directly fed to large language models, they do not handle it well. Existing work only utilizes LLMs for testing relatively small programs or unit testing.

Based on this understanding, our approach does not predominately rely on LLMs. We simply feed natural language descriptions of program documents to them, which is clearly within their capabilities. Then, considering the limitations of test cases generated by the LLM, we incorporate symbolic execution afterwards to generate more comprehensive test cases.

7. Conclusion

In this paper, we have proposed a novel program-context-guided symbolic execution framework LangSym, which can automatically extract knowledge from the user manual and use them to guide the symbolic execution. We have conducted extensive experiments on 103 Coreutils programs and 7 larger programs. The results showed that our approach achieved a significant improvement in code coverage compared to existing strategies in symbolic execution.

This work essentially demonstrates a new way to utilize LLMs for software engineering tasks (test case generation in particular), i.e., we show that a synergy of LLMs and more traditional program analysis approaches (e.g., symbolic execution) may give rise to new baselines for such tasks. In the future, we plan to explore more to unlock the potential of LLMs in software development.

Acknowledgment

We are thankful to the anonymous reviewers for their helpful comments. This work is supported by the National Natural Science Foundation of China (Grants #62025202, #62172199). T. Chen is partially supported by overseas grants of the State Key Laboratory of Novel Software Technology under Grants #KFKT2022A03 and #KFKT2023A04. Jingwei Xu (jingweix@nju.edu.cn) is the corresponding author.

References

[1] Srinivas Nidhra and Jagruthi Dondeti. Black box and white box testing techniques-a literature review. *International Journal of Embedded Systems and Applications (IJESA)*, 2(2):29–50, 2012.

[2] James C King. Symbolic execution and program testing. *Communications of the ACM*, 19(7):385–394, 1976.

[3] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR)*, 51(3):1–39, 2018.

[4] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, volume 8, pages 209–224, 2008.

[5] Cristian Cadar, Vijay Ganesh, Peter M Pawlowski, David L Dill, and Dawson R Engler. Exe: Automatically generating inputs of death. *ACM Transactions on Information and System Security (TISSEC)*, 12(2):1–38, 2008.

[6] You Li, Zhendong Su, Linzhang Wang, and Xuandong Li. Steering symbolic execution to less traveled paths. *ACM SigPlan Notices*, 48(10):19–32, 2013.

[7] Chao-Chun Yeh, Han-Lin Lu, Jia-Jun Yeh, and Shih-Kun Huang. Path exploration based on monte carlo tree search for symbolic execution. In *2017 Conference on Technologies and Applications of Artificial Intelligence (TAAI)*, pages 33–37. IEEE, 2017.

[8] Kasper Luckow, Corina S Păsăreanu, and Willem Visser. Monte carlo tree search for finding costly paths in programs. In *Software Engineering and Formal Methods: 16th International Conference, SEFM 2018, Held as Part of STAF 2018, Toulouse, France, June 27–29, 2018, Proceedings 16*, pages 123–138. Springer, 2018.

[9] Sooyoung Cha, Seongjoon Hong, Jiseong Bak, Jingyoung Kim, Junhee Lee, and Hakjoo Oh. Enhancing dynamic symbolic execution by automatically learning search heuristics. *IEEE Transactions on Software Engineering*, 48(9):3640–3663, 2021.

[10] Jingxuan He, Gishor Sivanrupan, Petar Tsankov, and Martin Vechev. Learning to explore paths for symbolic execution. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2526–2540, 2021.

[11] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[12] Luciano Floridi and Massimo Chiriatti. Gpt-3: Its nature, scope, limits, and consequences. *Minds and Machines*, 30:681–694, 2020.

[13] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. Large language models for software engineering: Survey and open problems. *arXiv preprint arXiv:2310.03533*, 2023.

[14] Ipek Ozkaya. Application of large language models to software engineering tasks: Opportunities, risks, and implications. *IEEE Software*, 40(3):4–8, 2023.

[15] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[16] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

[17] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[18] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*, pages 1–7, 2022.

[19] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36, 2024.

[20] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 2023.

[21] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. No more manual tests? evaluating and improving chatgpt for unit test generation. *arXiv preprint arXiv:2305.04207*, 2023.

[22] Shengcheng Yu, Chunrong Fang, Yuchen Ling, Chentian Wu, and Zhenyu Chen. Llm for test script generation and migration: Challenges, capabilities, and opportunities. In *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)*, pages 206–217. IEEE, 2023.

[23] Yaroslav Oliinyk, Michael Scott, Ryan Tsang, Chongzhou Fang, Houman Homayoun, et al. Fuzzing busybox: Leveraging llm and crash reuse for embedded bug unearthing. *arXiv preprint arXiv:2403.03897*, 2024.

[24] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. Inferfix: End-to-end program repair with llms. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1646–1656, 2023.

[25] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[26] Patrice Godefroid, Nils Klarlund, and Koushik Sen. Dart: Directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation*, pages 213–223, 2005.

[27] Patrice Godefroid, Michael Y Levin, David A Molnar, et al. Automated whitebox fuzz testing. In *NDSS*, volume 8, pages 151–166, 2008.

[28] Nick Stephens, John Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. Driller: Augmenting fuzzing through selective symbolic execution. In *NDSS*, volume 16, pages 1–16, 2016.

[29] Rupak Majumdar and Koushik Sen. Hybrid concolic testing. In *29th International Conference on Software Engineering (ICSE'07)*, pages 416–426. IEEE, 2007.

[30] Brian S Pak. Hybrid fuzz testing: Discovering software bugs via fuzzing and symbolic execution. *School of Computer Science Carnegie Mellon University*, 2012.

[31] Mingzhe Wang, Jie Liang, Yuanliang Chen, Yu Jiang, Xun Jiao, Han Liu, Xibin Zhao, and Jiaguang Sun. Safl: increasing and accelerating testing coverage with symbolic execution and guided fuzzing. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*, pages 61–64, 2018.

[32] Li Zhang and Vrizlynn LL Thing. A hybrid symbolic execution assisted fuzzing method. In *TENCON 2017-2017 IEEE Region 10 Conference*, pages 822–825. IEEE, 2017.

[33] Saahil Ognawala, Thomas Hutzelmann, Eirini Psallida, and Alexander Pretschner. Improving function coverage with munch: a hybrid fuzzing and directed symbolic execution approach. In *Proceedings of the 33rd Annual ACM Symposium on Applied Computing*, pages 1475–1482, 2018.

[34] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. Code-aware prompting: A study of coverage guided test generation in regression setting using llm. *arXiv preprint arXiv:2402.00097*, 2024.

[35] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 919–931. IEEE, 2023.